

Порівняння десктоп- та веб-клієнта проєкту Remote File Folder Client (спрощений аналог Google Drive)

Уточкіна Яна

Етап: порівняльний аналіз двох підходів до клієнта
30 вересня 2026 р.

Анотація

Проєкт **Remote File Folder Client** постачається з двома клієнтами – нативним застосунком macOS (Swift/SwiftUI) та браузерним застосунком (React/TypeScript) – проти одного спільного Go API та бекенду PostgreSQL. Обидва клієнти вже мають власний окремий звіт з описом внутрішньої структури, прецедентів використання та покриття тестами. Цей звіт натомість порівнює їх безпосередньо: що в них спільне, де вони справді відрізняються, і які компроміси несе кожен підхід.

Зміст

1 Вступ	3
2 Спільні елементи	3
2.1 Єдиний контракт з бекендом	3
2.2 Той самий основний набір функцій	3
2.3 Той самий алгоритм синхронізації	3
2.4 Збережений курсор синхронізації	4
3 У чому вони відрізняються	4
3.1 Тригер синхронізації: безперервний проти разового	4
3.2 Доступ до теки: нативні шляхи проти піскового дескриптора .	4
3.3 Сортування й фільтрація: на клієнті проти на сервері	5
3.4 Зберігання локального стану	5
3.5 Мережевий стек	5

3.6	Поширення та охоплення	5
3.7	Модель конкурентності	6
4	Компроміси кожного підходу	6
4.1	Десктоп (Swift/SwiftUI, нативний macOS)	6
4.2	Веб (React/TypeScript, браузер)	6
5	Підсумкова таблиця	7
6	Висновок	8

1. Вступ

Обидва клієнти надають ту саму персональну робочу область файлів, звертаються до того самого API через HTTPS за тим самим публічним хостнеймом і відображають ті самі записи файлів. Вони не були створені незалежно одне від одного з подальшим порівнянням постфактум – функція синхронізації теки веб-клієнта є прямим, свідомим портом рушія синхронізації десктоп-клієнта, адаптованим до того, що реально може браузер. Цей зв'язок формує більшу частину подальшого викладу: обидва клієнти тісно узгоджені щодо того, *що* робить продукт, і відрізняються переважно тим, як кожна платформа дозволяє це робити.

2. Спільні елементи

2.1 Єдиний контракт з бекендом

Обидва клієнти говорять точно тим самим протоколом з тим самим Go API: реєстрація та вхід за логіном/паролем із JWT access/refresh токенами, та сама структура запису файлу (ім'я, розширення, розмір, часові позначки створення/модифікації, автор вивантаження, редактор), ті самі ендпойнти вивантаження (одноразовий multipart для малих файлів, chunked-сесія initiate/chunk/complete для великих, обидва обмежені, щоб не перевищувати ліміт розміру тіла запиту Cloudflare Tunnel), та той самий ендпоінт попереднього перегляду вмісту `.java/.png`. Контракт з бекендом жодного з клієнтів не є підмножиною чи надмножиною іншого – це один і той самий контракт, яким користуються два різні фронтенди.

2.2 Той самий основний набір функцій

Реєстрація/вхід, перегляд файлів із сортованими/фільтрованими атрибутами, перегляд `.java` як тексту та `.png` як зображення, вивантаження (через вибір файлу і шляхом перетягування файлів з робочого столу/браузера), завантаження та видалення – кожна з цих функцій існує ідентично в обох клієнтах, відрізняючись лише тим, як кожен інструментарій інтерфейсу платформи виражає її (подання SwiftUI проти компонентів React).

2.3 Той самий алгоритм синхронізації

Синхронізація теки виконує той самий алгоритм з обох сторін: сканування теки для плоского (нерекурсивного) маніфесту імені/розміру/SHA-256 для кожного файлу, надсилання його разом з останньою синхронізованою версією на `POST /api/sync/diff`, застосування отриманих дій `download/delete_local/conflict`, а потім незалежне вивантаження нових

чи змінених локальних файлів і видалення віддалених копій локально видалених. Обидві реалізації виключають ті самі три випадки з цього фінального проходу в межах одного циклу – файл, уже позначений як конфлікт, файл, який сервер щойно велів видалити, і файл, щойно завантажений у цьому самому циклі – оскільки пропуск будь-якого з трьох дозволяє проходу неправильно прочитати побічний ефект цього самого циклу як справжню локальну зміну. Обидва розв’язують конфлікт тими самими трьома варіантами (Keep Local, Keep Remote, Keep Both), і обидва називають копію “keep both” однаково: Name (conflict copy, YYYY-MM-DD).ext.

2.4 Збережений курсор синхронізації

Обидва клієнти пам’ятають номер останньої синхронізованої версії та запис по кожному файлу (віддалений ID, розмір, хеш) між перезапусками, тож синхронізації не доводиться щоразу починати з нуля. Для цього використовуються різні технології зберігання (див. §3), але дані, які зберігаються, і мета їх зберігання ідентичні.

3. У чому вони відрізняються

3.1 Тригер синхронізації: безперервний проти разового

Десктоп-клієнт спостерігає за обраною текою у фоновому режимі через FSEvents і додатково опитує сервер кожні 45 секунд, тож зміна з будь-якого боку помічається за мить без будь-яких дій користувача. Браузер узагалі не може безперервно спостерігати за локальною текою у фоновому режимі – тож синхронізація веб-клієнта є одним разовим кліком кнопки **Sync now**: один цикл узгодження за запитом, і між кліками нічого не відбувається, незалежно від того, що змінюється з будь-якого боку.

3.2 Доступ до теки: нативні шляхи проти піскового дескриптора

Десктоп-клієнт працює зі звичайними шляхами файлової системи через FileManager – після обрання теки її можна вільно й безстроково читати й записувати, без подальших запитів дозволу. Веб-клієнт може дістатися лише теки, яку користувач явно надав через File System Access API браузера, який повертає непрозорий, пісочний дескриптор, а не шлях; дозвіл для цього дескриптора може втратити чинність між сесіями і може бути повторно запитаний лише зсередини справжнього кліку користувача, а не автоматично при завантаженні сторінки. Сам API доступний лише в браузерах на основі Chromium (Chrome, Edge, Brave) – у Firefox та Safari немає жодного еквівалента, тож уся функція синхронізації там недоступна, то-

ді як решта функцій (перегляд, вивантаження, завантаження, видалення) працює в будь-якому браузері.

3.3 Сортування й фільтрація: на клієнті проти на сервері

Десктоп-клієнт завантажує повний список файлів користувача один раз (до фіксованої межі) і сортує/фільтрує його локально в пам'яті при кожній взаємодії з інтерфейсом. Веб-клієнт натомість надсилає активний стовпець сортування, порядок і фільтр розширення на сервер як параметри запиту й повторно завантажує пагіновану сторінку при кожній зміні, ніколи не тримаючи в пам'яті більш ніж віконний (віртуалізований) набір рядків одночасно. Обидва дають той самий видимий результат – вони просто розміщують роботу сортування/фільтрації на протилежних кінцях з'єднання.

3.4 Зберігання локального стану

Десктоп-клієнт тримає курсор синхронізації та відомі записи файлів в одному JSON-файлі в Application Support, який читається й записується через Codable. Веб-клієнт тримає такі самі дані в IndexedDB, за невеликою абстракцією ключ-значення, оскільки сторінка браузера не має власної файлової системи, куди можна записати звичайний файл – і там само зберігає дескриптор обраної теки, що взагалі можливо лише тому, що `FileSystemDirectoryHandle` придатний до структурного клонування.

3.5 Мережевий стек

Десктоп-клієнт використовує `URLSession` для кожного запиту, включно з прогресом `chunked`-вивантаження. Веб-клієнт використовує `fetch` для всього, крім звітування про прогрес `chunked`-вивантаження, яке виконує через `XMLHttpRequest`, оскільки `fetch` взагалі не має події прогресу вивантаження.

3.6 Поширення та охоплення

Десктоп-клієнт постачається як `.dmg`, потребує одноразового встановлення, працює лише на macOS і потребує повторного встановлення для отримання нової версії. Веб-клієнт не потребує жодного встановлення, працює на будь-якому пристрої з сучасним браузером і завжди є найновішою версією в момент завантаження сторінки – за ціну того, що саме функція синхронізації обмежена браузерами на основі Chromium, як описано вище.

3.7 Модель конкурентності

Оскільки синхронізацію десктоп-клієнта можуть одночасно запустити два незалежних джерела – опитування кожні 45 секунд та відкладений callback спостереження за текою – його процедура узгодження має захищатися від одночасного повторного запуску; без цього захисту два перетинних проходи, що читають і записують той самий стан, можуть кожен працювати зі застарілим знімком змін, що вже виконує інший. Веб-клієнт не має такого ризику за своєю конструкцією: прохід синхронізації завжди починається лише з одного кліку кнопки, тож немає другого тригера, з яким можна б було конкурувати.

4. Компроміси кожного підходу

4.1 Десктоп (Swift/SwiftUI, нативний macOS)

Переваги:

- Безперервна фонові синхронізація – зміни з будь-якого боку з'являються без жодних дій користувача;
- Необмежений, бездозвільний доступ до будь-якої локальної теки після її обрання;
- Нативна інтеграція з ОС: Keychain для зберігання токенів, нативні діалоги вибору файлів, без пісочниці браузера, яку потрібно обходити;
- Жодних винятків через сумісність браузерів – кожна функція працює однаково щоразу.

Недоліки:

- Лише для macOS, і потребує встановлення (та періодичного повторного встановлення) .dmg;
- Сортування/фільтрація тримає весь список файлів користувача в пам'яті й обробляє його на клієнті, що нормально в масштабі персонального диска, але не масштабується безмежно;
- Модель безперервної синхронізації архітектурно складніша, і ця складність уже спричинила три реальні помилки конкурентності/застарілості під час розробки (витік стану при перемиканні теки, перегони через повторний вхід у процедуру узгодження та видалення файлу в тому самому циклі, що й завантаження), які довелося знайти та виправити.

4.2 Веб (React/TypeScript, браузер)

Переваги:

- Без встановлення, працює на будь-якій ОС із сучасним браузером, завжди найновіша версія;

- Серверне сортування/фільтрація та пагінація масштабуються до значно більшої кількості файлів, ніколи не тримаючи весь список на клієнті;
- Суттєво простіша модель конкурентності синхронізації – одна кнопка, один прохід, немає фонових тригерів, з яким можна конкурувати;
- Успадкував уже налагоджений алгоритм синхронізації, тож запустився з усіма трьома важко здобутими винятками застарілості десктоп-клієнта вже на місці, а не з необхідністю повторно виявляти їх самостійно.

Недоліки:

- Жодної фонові синхронізації – зміна, зроблена деінде, локально невидима, доки користувач не згадає повторно клікнути **Sync now**;
- Саме функція синхронізації працює лише в браузері на основі Chromium; вона повністю відсутня у Firefox та Safari;
- Дозвіл на теку не постійний – він може втратити чинність між сесіями і має бути наданий знову зі свіжого кліку користувача, що додає тертя, якого немає в нативному застосунку;
- Збережений стан живе в сховищі браузера (IndexedDB), яке підпорядковується власній політиці витіснення сховища браузера під тиском дискового простору, на відміну від власних файлів нативного застосунку на диску.

5. Підсумкова таблиця

Контракт з бекендом, основний набір функцій та алгоритм синхронізації/модель конфліктів ідентичні для обох клієнтів (див. розд. 2), тож не включені в таблицю нижче – таблиця 1 стисло зводить лише ті аспекти, де обидва клієнти справді відрізняються.

Табл. 1: Десктоп проти веб-клієнта, одним поглядом

Аспект	Десктоп (Swift/SwiftUI)	Веб (React/TypeScript)
Платформа	Лише macOS	Будь-яка ОС із сучасним браузером
Тригер синхронізації	Безперервний – FSEvents + опитування кожні 45с	Разовий – один клік, один прохід
Доступ до теки	Нативний шлях, бездозвільний після обрання	Пісочний дескриптор; дозвіл може втратити чинність
Доступність синхронізації	Завжди (нативний застосунок)	Лише Chromium – відсутня у Firefox/Safari
Сортування й фільтрація	На клієнті, над обмеженим повним завантаженням	На сервері, через параметри пагінованого запиту
Зберігання стану	JSON-файл	IndexedDB, за абстракцією ключ-значення

Табл. 1: Десктоп проти веб-клієнта, одним поглядом

Аспект	Десктоп (Swift/SwiftUI)	Веб (React/TypeScript)
Мережевий стек	URLSession	fetch + XMLHttpRequest
Поширення	Встановлюваний .dmg; повторне встановлення	Без встановлення; завжди найновіша версія
Ризик конкурентності	Потребує явного захисту від повторного входу	Відсутній за конструкцією

6. Висновок

Жоден з клієнтів не є просто кращою версією іншого – вони обмінюють те саме обмеження (браузер не може спостерігати за текою у фоновому режимі) на два різні набори переваг. Десктоп-клієнт – це місце, де справді доречна безперервна, безтертя синхронізація теки, за ціну нативного, встановлюваного застосунку лише для macOS. Веб-клієнт – це місце, де доречний доступ без встановлення, кросплатформний і завжди найновіший, за ціну того, що синхронізація є разовою дією, обмеженою специфічним для Chromium API браузера. Велика область перетину – той самий контракт з бекендом, той самий основний набір функцій, а саме для синхронізації – точно той самий алгоритм і модель конфліктів – це те, що робить обидва схожими на один продукт із двома входними дверима, а не на два непов'язані застосунки, які просто випадково користуються спільним сервером.